Technical Blocker Mitigation — Defender for Cloud Apps

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Technical Blocker Mitigation: Defender for Cloud Apps 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Boundary . . . 4

Effort . . . 5

1. Solution Overview — Technical Blocker Mitigation: Defender for Cloud Apps 5

What this solution is . . . 5

How Defender for Cloud Apps signal actually works (so you can find the break) . . . 6

The standard deployment model (applied to a blocker) . . . 6

Scope . . . 6

Out of scope . . . 6

Connections and authentication used . . . 6

2. Requirements Guide — Technical Blocker Mitigation: Defender for Cloud Apps 7

2.1 Read-only access to the Defender portal and Graph . . . 7

2.2 The affected app / connector / policy subset defined . . . 7

2.3 Knowledge of the Cloud Discovery source . . . 7

2.4 The Conditional Access App Control policy identified . . . 8

2.5 A test session for session-control verification . . . 8

2.6 A place to store the diagnostic output and support bundle . . . 8

Checklist . . . 8

Global Requirements — Defender (category-level) 9

G1. Microsoft 365 / Entra tenant . . . 9

G2. Defender licensing . . . 9

G3. Administrative roles (least privilege) . . . 9

G4. Onboarding reach to endpoints / identity / workloads . . . 9

G5. Network access to Defender service endpoints . . . 9

G6. A pilot ring . . . 10

G7. Log Analytics workspace (where required) . . . 10

G8. Response governance . . . 10

3. Implementation Guide — Technical Blocker Mitigation: Defender for Cloud Apps 10

Naming (set once, reuse verbatim) . . . 10

Step 1 — Assess: pinpoint the blocker (read-only) . . . 11

Step 2 — Configure: apply the one targeted fix (manual, documented) . . . 11

Step 3 — Pilot: re-test the fix on the affected apps/policies only . . . 11

Step 4 — Validate: confirm the blocker is cleared . . . 12
Step 5 — Expand: resume the stalled rollout . . . 12
Step 6 — Escalate (only if it is a service issue, not your config) . . . 12
5. Testing Guide — Technical Blocker Mitigation: Defender for Cloud Apps 12
Test 1 — Blocker pinpointed (baseline captured) . . . 13
Test 2 — Connector connected (activity signal) . . . 13
Test 3 — Cloud Discovery data . . . 13
Test 4 — Conditional Access App Control present and scoped . . . 13
Test 5 — Session control engages on a test session . . . 13
Test 6 — Licensing present . . . 13
Test 7 — Before/after evidence produced . . . 13
Test 8 — Escalation bundle valid (only if escalating) . . . 14
6. Troubleshooting Guide — Technical Blocker Mitigation: Defender for Cloud Apps 14
Connector authentication failure . . . 14
Cloud Discovery no-data . . . 14
Session policy / CA App Control not engaging . . . 14
Licensing gap . . . 15
Script cannot authenticate . . . 15
When to escalate to Microsoft support . . . 15
7. Best Practices — Technical Blocker Mitigation: Defender for Cloud Apps 15
Diagnose before you touch a connector or a CA policy . . . 15
Change one thing, on the affected apps, monitor-first, then verify . . . 16
Session control is the whole Conditional Access chain . . . 16
Keep enforcement approval-gated . . . 16
Keep everything read-only until you deliberately fix . . . 16
Let connector, discovery, and session signal warm up . . . 16
Keep licensing and preview facts current . . . 16
Escalate with evidence, not a description . . . 16
Where to go next . . . 16
8. Deliverables Summary — Technical Blocker Mitigation: Defender for Cloud Apps 17
Documentation . . . 17
PowerShell scripts (Graph advanced-hunting KQL + Conditional Access reads — all read-only) . . . 17
What the buyer receives . . . 18
Outcome . . . 18
Appendix A — PowerShell Scripts 19

Executive Summary — Technical Blocker Mitigation: Defender for Cloud Apps

White-label · Defender

A Defender for Cloud Apps (MDCA) rollout almost always stalls on the same short list of blockers: **app-connector authentication failures** (an API connector shows error/disconnected after a token or permission change), **Cloud Discovery returning no data** (the discovery source is not feeding the service), or **session policies / Conditional Access App Control not engaging** (traffic never routes through the MDCA reverse proxy, so inline controls do nothing) — plus the usual **licensing gap**. Left undiagnosed, MDCA looks connected but sees or controls nothing. This solution is a **diagnostic-first playbook**: read-only discovery that pinpoints *which* blocker you actually have, the **targeted, documented manual fix**, and the verification that proves the connector, discovery, or session control is working — so a stuck rollout gets moving again fast.

This is a **run-it-yourself** package written for an administrator with no prior deep MDCA troubleshooting experience. Every script is **read-only** — it reads connector/activity signal, Conditional Access policy state, and cloud-app telemetry; it never changes a connector, a policy, or a session control. Any fix is a **documented manual step** you apply deliberately, on the affected app / policy subset first.

What you get

- A plain-language **Solution Overview** of the MDCA signal chain — activation, app connectors, Cloud Discovery, session / Conditional Access App Control — and where each one breaks.
- A **Requirements Guide** with verify / implement-if-missing for the diagnostic-specific prerequisites.
- An **Implementation Guide** that walks the diagnostic loop end to end — pinpoint the blocker, apply the targeted fix, re-test on the affected apps/policies, confirm engaging, resume the rollout.
- Five **read-only PowerShell scripts** (Microsoft Graph advanced-hunting KQL + Conditional Access policy reads) for discovery, Conditional Access App Control analysis, blocker validation, a status report, and a support-ready data bundle.
- **Testing** and **Troubleshooting** guides that map every common symptom to its cause and fix.

What this delivers

A previously stuck Defender for Cloud Apps rollout **moving again** — the blocker diagnosed, the targeted fix applied and verified on the affected apps/policies, and, if the root cause is a Microsoft service issue rather than your configuration, a **support-ready diagnostic bundle** and clear escalation criteria.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — assess with read-only discovery to pinpoint the blocker, apply the targeted fix, pilot the fix on the affected app / policy subset, validate the connector / discovery / session control engages, then expand by resuming the stalled rollout.

Boundary

This is a **diagnostic and troubleshooting** solution. It gets a *stalled* MDCA deployment unstuck; it does **not** replace a full CASB build or a cloud-app DLP engagement (see the adjacent *Cloud App Security DLP* solution). All scripts are **read-only**. Enforcing session controls (block, restrict, quarantine) stays **approval-gated** and is the SOC's call; incident-response and licensing determinations remain the customer's functions — this guide provides the technical diagnosis and the documented fix, not the decision.

Effort

A competent administrator can run discovery, identify the blocker, apply the documented fix, and verify the connector / discovery / session control engages for a single blocker class in a focused session. Reconnecting all app connectors and re-scoping session policies scales with the number of apps.

1. Solution Overview — Technical Blocker Mitigation: Defender for Cloud Apps

What this solution is

This is a **diagnostic and troubleshooting playbook** for a **stalled Microsoft Defender for Cloud Apps (MDCA) rollout**. It assumes you have started connecting apps, turning on discovery, or wiring session controls, and hit a wall — a connector that will not stay authenticated, a discovery dashboard with no data, or session policies that never engage. Its job is to answer three questions quickly and in order:

1. **Which blocker do I actually have?** (read-only discovery of connector, discovery, and session-control state — not guesswork)
2. **What is the exact, documented fix?** (a targeted manual step, applied deliberately)
3. **Is the connector / discovery / session control now working?** (verification before you resume)

It targets the four blocker classes that stall almost every MDCA deployment:

• **Connector authentication failures** — an **app connector** (Microsoft 365, AWS, Salesforce, ServiceNow, etc.) shows **error / disconnected** after a token expiry, a revoked app registration, or a changed permission.

• **Cloud Discovery no-data** — the discovery **dashboard is empty** because the discovery source is not feeding MDCA (the **Defender for Endpoint integration** is off, or log-collector / secure-web-gateway / manual log uploads are not arriving).

• **Session policy / Conditional Access App Control not engaging** — inline controls do nothing because sessions never route through the MDCA **reverse proxy**: the **Conditional Access App Control** policy is missing/misscoped, the app is not onboarded to session control, or the browser/session is not being redirected.

• **Licensing gap** — the MDCA plan (and, for CA App Control, the required Entra plan) is not assigned. *Verify current plan requirements in your tenant.*

How Defender for Cloud Apps signal actually works (so you can find the break)

MDCA has a signal chain, and a blocker is always a break in one specific link:

1. **Licensing/activation** — the tenant has the MDCA plan and the portal is activated.
2. **App connectors** — API connectors to SaaS apps are **authenticated** and connected, pulling activity and enabling governance.
3. **Cloud Discovery** — a discovery source (the **MDE integration** for automatic device-based discovery, or uploaded logs) feeds the discovery dashboard so shadow-IT is visible.
4. **Session / Conditional Access App Control** — Entra **Conditional Access** routes selected sessions to the MDCA reverse proxy, the target app is **onboarded to session control**, and the session policy applies inline (monitor / restrict / block).
5. **Verification** — the connector shows **connected**, discovery shows data, session policies engage on a test session, and advanced-hunting `CloudAppEvents` streams.

Find the earliest broken link and you have found the blocker. Discovery reports the state of each link so you are not guessing.

The standard deployment model (applied to a blocker)

> ***Assess*** → ***Configure*** → ***Pilot*** → ***Validate*** → ***Expand. Assess*** *— run read-only discovery and the Conditional Access App Control analysis to pinpoint the earliest broken link.* ***Configure*** *— apply the single documented fix for that blocker (a manual portal step, not a script action).* ***Pilot*** *— re-test on the* ***affected app / policy subset*** *only, in monitor mode.* ***Validate*** *— confirm the connector / discovery / session control engages.* ***Expand*** *— resume the stalled rollout to the remaining apps/policies.*

Scope

- Diagnosing and clearing the four blocker classes above for a stalled MDCA rollout.
- Read-only discovery, Conditional Access App Control analysis, validation, status reporting, and a support-ready bundle.
- Documented manual fixes for each blocker, applied affected-app-first and monitor-first.
- Clear escalation criteria and the exact data to collect when the root cause is a Microsoft service issue.

Out of scope

- **A full CASB / cloud-app DLP build** — that is the adjacent *Cloud App Security DLP* solution; this one unblocks a rollout already in progress.
- **Enforcing session controls in blocking mode** — block / restrict / quarantine stay **approval-gated** and are the SOC's decision; diagnostics run monitor-first.
- **Broader identity / Conditional Access design** — see the adjacent Entra Conditional Access solution; here CA is diagnosed only as it relates to App Control.
- **Licensing determinations** — the guide tells you *which* plan a feature needs and how to verify it; assigning it is the customer's licensing function.

Connections and authentication used

• The **unified Defender portal** (`security.microsoft.com`) — **Cloud apps** → **App connectors / Cloud discovery / Policies**, and activity log.

• **Microsoft Graph** — read-only **Conditional Access policy** reads (to analyze App Control) and the **advanced-hunting** `CloudAppEvents` table.

• **Entra Conditional Access** (`entra.microsoft.com`) — the policy that routes sessions to the MDCA reverse proxy (read-only here).

• Least-privilege roles: **Security Reader** / **Global Reader** for all read-only diagnostics; **Security Administrator** (and Conditional Access Administrator for a CA fix) only to apply a fix.

2. Requirements Guide — Technical Blocker Mitigation: Defender for Cloud Apps

Verify **every** requirement before diagnosing. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (tenant, Defender licensing, admin roles, onboarding reach, network access, pilot ring, workspace, response governance) are in `global-requirements-defender.md` — do not repeat them; the items below are **specific to running the MDCA blocker diagnostics**. Objects you create are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

2.1 Read-only access to the Defender portal and Graph

Description: Security Reader / **Global Reader** in the Defender portal and consent to the read-only Microsoft Graph scopes (`ThreatHunting.Read.All`, `Policy.Read.All` for Conditional Access reads). **Why:** The diagnostics read connector/activity signal (`CloudAppEvents`) and Conditional Access App Control policy state — all read-only — to pinpoint the blocker. **Verify:** Sign in to `security.microsoft.com`; confirm **Cloud apps** → **App connectors / Cloud discovery** load. `TBMMCA-Discovery.ps1` reports the roles/scopes it resolved. **Implement-if-missing:** Assign **Security/Global Reader**; grant admin consent to `ThreatHunting.Read.All` and `Policy.Read.All`.

2.2 The affected app / connector / policy subset defined

Description: A named subset of the apps, connectors, or session policies exhibiting the blocker — the ones you will re-test the fix against before resuming. Name it `MDCA Blocker Triage – Affected Apps – Diagnose`. **Why:** Fixes are validated on the affected subset first (per `global-requirements-defender.md` G6), in monitor mode, before touching the wider estate. **Verify:** The list of affected apps/connectors/policies exists; `TBMMCA-Validate.ps1` checks their signal. **Implement-if-missing:** Document the affected apps/connectors/policies as `MDCA Blocker Triage – Affected Apps – Diagnose`.

2.3 Knowledge of the Cloud Discovery source

Description: Know how Cloud Discovery is meant to be fed — the automatic **Defender for Endpoint integration**, a **log collector**, a **secure web gateway** feed, or **manual log upload**. **Why:** "Discovery no-data" is diagnosed differently per source; you cannot fix a feed you have not identified. **Verify:** `security.microsoft.com` → **Cloud apps** → **Cloud discovery** → **Settings**; confirm the intended

source. `TBMMCA-Discovery.ps1` reports whether device-based discovery signal is present. **Implement-if-missing:** Decide/confirm the discovery source with whoever owns it; for MDE-based discovery, confirm endpoints are onboarded to MDE.

2.4 The Conditional Access App Control policy identified

Description: Know which Entra **Conditional Access** policy is meant to route sessions to the MDCA reverse proxy — e.g. `Use Conditional Access App Control – Pilot Apps – Diagnose` — and which apps/users it targets. **Why:** Session policies never engage unless a CA policy with the **Conditional Access App Control** session control applies to the session; this is the top session-control blocker. **Verify:** `entra.microsoft.com` → **Protection** → **Conditional Access** → **Policies**; `TBMMCA-ConditionalAccess.ps1` lists policies whose session controls use App Control. *Verify current CA App Control configuration in your tenant.* **Implement-if-missing:** Identify or plan the CA policy `Use Conditional Access App Control – Pilot Apps – Diagnose` for the fix step.

2.5 A test session for session-control verification

Description: A pilot user and a browser session you can use to exercise a session-controlled app and confirm the MDCA reverse proxy engages. **Why:** Session-control verification needs a live session routed through the proxy so you can confirm the inline control applies (or see why it does not). **Verify:** A pilot user in scope of the CA App Control policy can sign in to the target app in a browser; the session shows the MDCA proxy suffix where applicable. **Implement-if-missing:** Nominate a pilot user in the affected subset and a test browser. *Verify current session-control behavior in your tenant.*

2.6 A place to store the diagnostic output and support bundle

Description: A secure location for the JSON snapshots the scripts write and for the support bundle `MDCA Support Bundle – <case> – Escalate`. **Why:** The before/after snapshots prove the blocker cleared, and the bundle is what you attach to a Microsoft support case. **Verify:** A working folder exists with least-privilege access; `TBMMCA-Report.ps1` writes there. **Implement-if-missing:** Create a restricted working folder; do not store tenant/app identifiers in shared/unsecured locations.

Checklist

- Security/Global Reader + `ThreatHunting.Read.All` + `Policy.Read.All` — `TBMMCA-Discovery.ps1`
- Affected app/connector/policy subset `MDCA Blocker Triage – Affected Apps – Diagnose` documented
- Cloud Discovery source known (MDE integration / log collector / SWG / manual)
- CA App Control policy `Use Conditional Access App Control – Pilot Apps – Diagnose` identified — `TBMMCA-ConditionalAccess.ps1`
- Test session (pilot user + browser) ready for session-control verification
- Secure working folder for snapshots and the support bundle

Global Requirements — Defender (category-level)

These requirements apply to every solution in the Defender category (Microsoft Defender XDR and Defender for Cloud). Published once at the category level so listings stay small. Confirm before deploying any Defender solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Defender changes touch detection and response for the whole tenant. Onboard in a ring (pilot devices/users first), keep automated response in an approval/simulate posture until trusted, and never disable an existing control to "test."*

G1. Microsoft 365 / Entra tenant

Description: An active Microsoft 365 tenant with Entra ID. Why: Defender XDR services and the unified portal (security.microsoft.com) are tenant workloads. Verify: Sign in to `https://security.microsoft.com` as an administrator. Implement: Provision a Microsoft 365 tenant and confirm Entra ID identities exist.

G2. Defender licensing

Description: The Defender plans for the pillars in scope — e.g. Defender for Endpoint P1/P2, Defender for Identity, Defender for Office 365 P1/P2, Defender for Cloud Apps, and Defender for Cloud plans. Why: Each pillar/feature is license-gated; onboarding without the plan fails or silently limits features. Verify: Admin center -> `Billing -> Licenses` (or the Defender portal). *Confirm the exact plan each feature needs — verify current availability in your tenant.* Implement: Assign the required plans (often bundled in M365 E5 / E5 Security) before onboarding.

G3. Administrative roles (least privilege)

Description: Security Administrator (and Global Administrator for first-time service activation). Why: Onboarding, RBAC, and policy changes in the Defender portal require these roles. Verify: Entra -> `Roles and administrators`; confirm Security Administrator. Implement: Assign Security Administrator; reserve Global Administrator for one-time activations.

G4. Onboarding reach to endpoints / identity / workloads

Description: A management channel to onboard the pillars — Intune/Configuration Manager/GPO/script for endpoints, domain controllers/AD FS for Identity sensors, and cloud connectors for Defender for Cloud. Why: Each pillar collects signal from a source that must be reachable and instrumented. Verify: Endpoints are managed (Intune/CM); DCs are reachable; cloud subscriptions are connected. Run the solution's discovery script. Implement: Establish the onboarding channel per pillar (see the relevant catalog endpoint/identity solutions).

G5. Network access to Defender service endpoints

Description: Outbound HTTPS to the Microsoft Defender / Microsoft 365 service URLs from onboarded assets. Why: Sensors and agents report to the cloud service; blocked endpoints break onboarding and detections. Verify: Confirm the documented Defender endpoint URLs/IP ranges are allowed. *Verify current endpoint list.* Implement: Allow the Defender/M365 endpoints through proxy/firewall.

G6. A pilot ring

Description: A defined set of pilot devices/users/subscriptions to onboard first. Why: XDR onboarding and automated response are validated on a ring before tenant-wide rollout. Verify: A pilot group exists in Entra/Intune. Implement: Create the pilot ring before enabling anything that acts automatically.

G7. Log Analytics workspace (where required)

Description: A Log Analytics workspace for Defender for Cloud / advanced hunting export where used. Why: Some Defender data and continuous export land in a workspace. Verify: A workspace exists and is access-controlled. Run the discovery script. Implement: Provision a workspace (or reuse the SIEM/observability one) with least-privilege access.

G8. Response governance

Description: An agreed position on automated response (auto-remediation levels, approval requirements). Why: XDR/AIR can take disruptive actions (isolate, delete, disable); the organization must decide the guardrails. Verify: A documented response policy exists; automation is in an approval/simulate posture until trusted. Implement: Define response guardrails; keep AIR/automation approval-gated at first. *Incident-response decisions are the SOC/security team's; this guide covers the technical configuration.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Technical Blocker Mitigation: Defender for Cloud Apps

Follow these steps **in order**. This is a **diagnostic loop**, not a build: you assess to pinpoint the blocker, apply one targeted documented fix, re-test the affected apps/policies in monitor mode, and only then resume the rollout. All scripts are **read-only**; every *fix* is a manual step you apply deliberately. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

> ***Model:*** *Assess (pinpoint) → Configure (fix) → Pilot (re-test affected apps, monitor-first) → Validate (confirm engaging) → Expand (resume rollout). Keep session-control enforcement (block/restrict/quarantine) approval-gated and monitor-first throughout.*

Naming (set once, reuse verbatim)

- Affected app/connector/policy subset: `MDCA Blocker Triage – Affected Apps – Diagnose`
- Conditional Access App Control policy: `Use Conditional Access App Control – Pilot Apps – Diagnose`
- Advanced-hunting saved query (connector/activity check): `MDCA Connector Blocker Check – Tenant – Diagnose`
- Support bundle output: `MDCA Support Bundle – <case> – Escalate`
- Discovery snapshot file: `MDCA-blocker-discovery.json`

Step 1 — Assess: pinpoint the blocker (read-only)

1. Sign in to `security.microsoft.com` as **Security Reader** and open **Cloud apps** → **App connectors** and **Cloud discovery** → **Dashboard** to read current state.

2. Run `TBMMCA-Discovery.ps1`. It captures the signal chain: licensed MDCA plan, whether cloud-app activity signal is arriving (`CloudAppEvents` presence — a proxy for working connectors), whether device-based discovery signal is present, and a summary of Conditional Access policies that use **App Control**. It records the state to `MDCA-blocker-discovery.json`.

3. Run `TBMMCA-ConditionalAccess.ps1` to enumerate the CA policies whose session controls use **Conditional Access App Control**, their **enabled state**, and which apps/users they target.

4. Read both outputs top-down against the signal chain (Overview §"How signal actually works") and identify the **earliest broken link**:

• No plan / not activated → **licensing/activation blocker**

• Connector error/disconnected, no `CloudAppEvents` → **connector-auth blocker**

• Discovery dashboard empty → **discovery no-data blocker**

• No/misscoped CA App Control policy → **session-control blocker**

Step 2 — Configure: apply the one targeted fix (manual, documented)

Apply **only** the fix for the blocker you identified. Each is a deliberate manual step — the scripts never make these changes.

• **Licensing / activation:** Assign the MDCA plan (**Billing** → **Licenses**) and confirm the portal is activated. For CA App Control, confirm the required Entra plan. *Verify the exact plan requirements in your tenant.*

• **Connector authentication:** In **Cloud apps** → **App connectors**, reconnect the affected connector — re-consent the app registration, refresh the expired token, and confirm the connector account still has the required permissions in the SaaS app. Confirm status returns to **connected**.

• **Cloud Discovery no-data:** Confirm and repair the discovery source — turn on the **Defender for Endpoint integration** under **Cloud discovery** → **Settings** (and confirm devices are onboarded to MDE), or fix the log collector / secure-web-gateway feed / manual upload so logs arrive.

• **Session policy / CA App Control:** In `entra.microsoft.com` → **Conditional Access**, confirm the policy `Use Conditional Access App Control – Pilot Apps – Diagnose` is **enabled**, targets the affected users/apps, and its session control is set to **Use Conditional Access App Control** (monitor-only to start). In MDCA, confirm the target app is **onboarded to session control**.

Step 3 — Pilot: re-test the fix on the affected apps/policies only

1. Apply the fix scoped to `MDCA Blocker Triage – Affected Apps – Diagnose` only — not the whole estate — and keep session policies in **monitor** mode.

2. For a session-control fix, have the pilot user (Requirements 2.5) sign in to the target app in a browser and generate activity.

3. Give connector/discovery/session signal time to appear, then validate.

Step 4 — Validate: confirm the blocker is cleared

1. Run `TBMMCA-Validate.ps1`. It runs the saved query `MDCA Connector Blocker Check – Tenant – Diagnose` against `CloudAppEvents` for the affected apps and returns pass/fail for activity signal.

2. In the portal, confirm the connector shows **connected**, the discovery dashboard shows data, and the session policy shows a hit in the **activity log** for the test session.

3. Run `TBMMCA-ConditionalAccess.ps1` again to confirm the App Control policy is enabled and scoped.

4. Run `TBMMCA-Report.ps1` for the before/after evidence.

5. Complete the **Testing Guide** for the blocker class before expanding.

Step 5 — Expand: resume the stalled rollout

1. With the affected subset green, reconnect the remaining connectors / expand discovery / broaden the CA App Control scope in waves — still monitor-first for session controls.

2. Re-run `TBMMCA-Validate.ps1` after each wave and compare with `TBMMCA-Report.ps1`.

3. Only after monitoring is trusted, and with SOC approval, move any session policy from monitor to enforce. *That enforcement decision is the SOC's.*

Step 6 — Escalate (only if it is a service issue, not your config)

1. If the plan is assigned, the connector re-consented, the discovery source confirmed, and the CA App Control policy enabled and scoped, but the blocker persists, it may be a Microsoft service issue.

2. Run `TBMMCA-SupportBundle.ps1` to assemble `MDCA Support Bundle – <case> – Escalate` — the discovery snapshot, the CA App Control analysis, the affected apps/policies, and timestamps.

3. Open a Microsoft support case, attach the bundle, and reference the escalation criteria in the Troubleshooting Guide. *Incident-response decisions remain the SOC's; this is a service-support escalation.*

Order of operations

```
Assess (discovery + CA App Control analysis) → identify earliest broken link → apply the one targeted fix → re-test the affected apps monitor-first → validate engaging → resume rollout → escalate with a bundle only if it is a service issue
```

5. Testing Guide — Technical Blocker Mitigation: Defender for Cloud Apps

Run these checks after applying a fix on the affected subset (`MDCA Blocker Triage – Affected Apps – Diagnose`) and again after each expansion wave. Each test has a **pass/fail gate**; do not resume

the wider rollout until the affected subset passes the test for its blocker class. Session policies stay in **monitor** mode during testing. Objects are referenced by the exact names from the Implementation Guide.

Test 1 — Blocker pinpointed (baseline captured)

Procedure: Run `TBMMCA-Discovery.ps1` and confirm `MDCA-blocker-discovery.json` has the plan, activity-signal presence, discovery hint, and CA App Control summary populated. **Pass:** A single earliest broken link is identifiable; the affected apps are in the triage subset. **Fail:** Snapshot empty or access-denied — fix read access (Requirements 2.1) and re-run.

Test 2 — Connector connected (activity signal)

Procedure: Run `TBMMCA-Validate.ps1`; in **Cloud apps** → **App connectors** confirm the affected connector shows **connected**. **Pass:** The connector is connected and `CloudAppEvents` shows recent activity for the app. **Fail:** Connector error/disconnected or no activity — see Troubleshooting *Connector authentication failure*.

Test 3 — Cloud Discovery data

Procedure: In **Cloud apps** → **Cloud discovery** → **Dashboard**, confirm discovery data appears from the intended source. **Pass:** The discovery dashboard shows data. **Fail:** Dashboard empty — see Troubleshooting *Cloud Discovery no-data*.

Test 4 — Conditional Access App Control present and scoped

Procedure: Run `TBMMCA-ConditionalAccess.ps1`; confirm `Use Conditional Access App Control – Pilot Apps – Diagnose` is **enabled**, targets the affected users/apps, and its session control is **Use Conditional Access App Control**. **Pass:** A correctly scoped, enabled App Control policy exists. **Fail:** Missing / disabled / misscoped — see Troubleshooting *Session policy / CA App Control not engaging*.

Test 5 — Session control engages on a test session

Procedure: Have the pilot user sign in to the target app in a browser and generate activity; in the MDCA **activity log** confirm the session was proxied and the monitor-mode session policy recorded a hit. **Pass:** The test session routes through the MDCA reverse proxy and the session policy records the activity. **Fail:** Session not proxied — see Troubleshooting *Session policy / CA App Control not engaging*.

Test 6 — Licensing present

Procedure: Run `TBMMCA-Discovery.ps1` and confirm the MDCA plan (and the Entra plan for CA App Control) is reported as licensed. **Pass:** The required plans are present. *Verify current plan requirements in your tenant.* **Fail:** Plan missing — see Troubleshooting *Licensing gap*.

Test 7 — Before/after evidence produced

Procedure: Run `TBMMCA-Report.ps1` and confirm it emits a before/after comparison from the Step 1 snapshot versus current state. **Pass:** The report shows the blocker's link moved from broken to cleared.

Fail: No change captured — re-run discovery to refresh the baseline.

Test 8 — Escalation bundle valid (only if escalating)

Procedure: If the blocker persists after a correct fix, run `TBMMCA-SupportBundle.ps1` and confirm `MDCA Support Bundle – <case> – Escalate` contains the discovery snapshot, CA App Control analysis, affected apps/policies, and timestamps. **Pass:** A complete, attachable bundle exists. **Fail:** Bundle incomplete — re-run discovery / CA analysis first, then rebuild.

Exit gate

The affected subset passes the tests for its blocker class **before** you resume the wider rollout (Implementation Step 5). Re-run Tests 2–5 and 7 after each expansion wave and compare against the saved baseline. Move a session policy from monitor to enforce only with SOC approval.

6. Troubleshooting Guide — Technical Blocker Mitigation: Defender for Cloud Apps

The four blocker classes in **symptom** → **cause** → **fix** form, plus escalation. Run `TBMMCA-Discovery.ps1` and `TBMMCA-ConditionalAccess.ps1` first — most causes surface there. This *is* the troubleshooting solution, so this guide is the core reference.

Connector authentication failure

Symptom: An app connector in `MDCA Blocker Triage – Affected Apps – Diagnose` shows **error / disconnected**; no `CloudAppEvents` activity for the app. **Cause:** The connector's token expired, the app registration was revoked/changed, or the connector account lost the required permissions in the SaaS app. **Fix:** In **Cloud apps** → **App connectors**, reconnect the connector — re-consent the app registration, refresh the token, and confirm the connector account still has the required permissions in the SaaS app. Confirm status returns to **connected**, then re-run `TBMMCA-Validate.ps1`.

Cloud Discovery no-data

Symptom: Cloud discovery → **Dashboard** is empty; shadow-IT is invisible. **Cause:** The discovery source is not feeding MDCA — the **Defender for Endpoint integration** is off (or devices are not onboarded to MDE), or the log collector / secure-web-gateway feed / manual upload is not arriving. **Fix:** Confirm the intended source (Requirements 2.3). For device-based discovery, turn on the **MDE integration** under **Cloud discovery** → **Settings** and confirm devices are onboarded to MDE. For log-based discovery, repair the log collector / SWG feed or re-upload logs. Allow ingestion time, then re-check the dashboard.

Session policy / CA App Control not engaging

Symptom: A session policy never fires; the target app session is not proxied; inline controls do nothing. **Cause:** No enabled **Conditional Access App Control** policy applies to the session, the policy does not target the affected users/apps, or the app is not onboarded to session control. **Fix:** Run `TBMMCA-ConditionalAccess.ps1`; confirm `Use Conditional Access App Control – Pilot`

`Apps – Diagnose` is **enabled**, targets the affected users/apps, and its session control is **Use Conditional Access App Control**. In MDCA, confirm the app is **onboarded to session control**. Re-test with the pilot user's browser session and confirm a proxied session in the activity log.

Licensing gap

Symptom: MDCA connector / session-control options are missing or greyed out. **Cause:** The MDCA plan (or, for CA App Control, the required Entra plan) is not assigned. **Fix:** Assign the plan(s) (**Billing** → **Licenses**). *Verify the exact plan requirements — availability changes; confirm in your tenant.* Re-run `TBMMCA-Discovery.ps1`.

Script cannot authenticate

Symptom: `TBMMCA-*.ps1` fails on connect or returns access-denied. **Cause:** The Microsoft Graph modules are missing, `ThreatHunting.Read.All` / `Policy.Read.All` consent was not granted, or the account lacks Security/Global Reader. **Fix:** Let the script install the modules; sign in with `Connect-MgGraph` and consent to the read-only scopes; confirm Security/Global Reader. Re-run.

When to escalate to Microsoft support

Escalate — and attach `MDCA Support Bundle – <case> – Escalate` — when **all** of the following hold:

- The MDCA (and Entra, for App Control) plan is assigned.
- The affected connector was re-consented with the required permissions.
- The Cloud Discovery source is confirmed feeding the service.
- The CA App Control policy is enabled and correctly scoped, and the app is onboarded to session control.
- The correct documented fix was applied and warm-up time elapsed, yet the blocker persists.

In that case the root cause is likely a **service-side issue**, not your configuration. Run `TBMMCA-SupportBundle.ps1`, open the case, and attach the bundle. *Incident-response decisions remain the SOC's; this is a service-support escalation.*

7. Best Practices — Technical Blocker Mitigation: Defender for Cloud Apps

Field-proven guidance for diagnosing a stalled MDCA rollout cleanly, fixing the *right* thing, and not disrupting user sessions.

Diagnose before you touch a connector or a CA policy

- Run `TBMMCA-Discovery.ps1` and `TBMMCA-ConditionalAccess.ps1` **first**. Read the signal chain top-down; the earliest broken link is the blocker. A careless Conditional Access change can lock users out of an app — never edit CA on a hunch.

• A session policy that "never fires" is almost always a **Conditional Access App Control** scoping problem, not a broken policy — confirm the CA routing before touching the session policy.

Change one thing, on the affected apps, monitor-first, then verify

• Apply exactly **one** documented fix, scoped to `MDCA Blocker Triage – Affected Apps – Diagnose`, keep session policies in **monitor** mode, then run `TBMMCA-Validate.ps1`. One change at a time keeps cause and effect clear.

• Never resume the estate-wide rollout until the affected subset passes its test gate.

Session control is the whole Conditional Access chain

• Inline controls only work if Entra CA routes the session to the MDCA reverse proxy **and** the app is onboarded to session control. When it "does nothing," check the CA policy scope/enable-state and the app onboarding before assuming MDCA is broken.

Keep enforcement approval-gated

• Diagnostics and re-tests run **monitor-first**. Moving a session policy to block / restrict / quarantine is a **SOC decision** — keep it approval-gated so a fix does not become an outage.

Keep everything read-only until you deliberately fix

• Every script here is read-only, including the Conditional Access reads. Treat the *fix* (reconnect a connector, enable a discovery source, adjust a CA policy) as a separate, conscious action in the portal.

Let connector, discovery, and session signal warm up

• Connector re-consent, discovery ingestion, and `CloudAppEvents` populate over minutes to hours. Give a fix time before calling it a failure, then re-validate.

Keep licensing and preview facts current

• MDCA features are **plan-gated**, and CA App Control depends on the right Entra plan. Before concluding "it's broken," confirm the feature is even licensed — *verify current availability and plan requirements in your tenant.*

Escalate with evidence, not a description

• If the blocker survives a correct fix, attach `MDCA Support Bundle – <case> – Escalate` (with the CA App Control analysis) to the case. A complete bundle resolves cases far faster than a written symptom.

Where to go next

Once the rollout is unblocked, natural next steps in the catalog:

• **Cloud App Security DLP** to operate cloud-app data protection deeply.

- **Entra Conditional Access baseline** to harden the identity/CA layer that App Control rides on.
- **Defender XDR Deployment & Unified Onboarding** to correlate cloud-app alerts with endpoint, identity, and email signal in one portal.
- The **Sentinel** SOC/XDR integration to stream MDCA signal into a SIEM for long-term hunting.

8. Deliverables Summary — Technical Blocker Mitigation: Defender for Cloud Apps

Documentation

Document	Purpose
Executive Summary	The blockers, the diagnostic-first approach, the outcome, the boundary, effort
1. Solution Overview	The four MDCA blocker classes, the signal chain (connectors → discovery → session/CA App Control), where each breaks
2. Requirements Guide	Diagnostic-specific prerequisites with Verify / Implement-if-missing
Global Requirements (Defender)	Category-wide prerequisites, published once
3. Implementation Guide	The diagnostic loop — assess, fix, re-test the affected apps monitor-first, validate, resume, escalate
5. Testing Guide	Eight pass/fail tests, one per link in the signal chain
6. Troubleshooting Guide	Every blocker class in symptom → cause → fix, plus escalation criteria
7. Best Practices	Diagnose-first discipline, session-control chain, and where to go next

PowerShell scripts (Graph advanced-hunting KQL + Conditional Access reads — all read-only)

Script	What it does
`TBMMCA-Discovery.ps1`	Read-only signal-chain snapshot — licensed MDCA plan, cloud-app activity signal (`CloudAppEvents` as a proxy for working connectors), device-based discovery hint, and a summary of CA policies using App Control
`TBMMCA-ConditionalAccess.ps1`	Read-only analysis of Entra Conditional Access policies whose session controls use **Conditional Access App Control** — enabled state and which apps/users they target

Script	What it does
`TBMMCA-Validate.ps1`	Read-only pass/fail for the affected apps — runs the saved hunting query against `CloudAppEvents` to confirm connector/activity signal is flowing
`TBMMCA-Report.ps1`	Read-only before/after status report from the discovery baseline versus current state — the evidence the blocker cleared
`TBMMCA-SupportBundle.ps1`	Read-only assembly of the support-ready bundle (discovery snapshot, CA App Control analysis, affected apps/policies, timestamps) for a Microsoft support case

What the buyer receives

- This solution as a **Word** document and **PDF**.
- All five **read-only PowerShell scripts** (with embedded KQL) in a single ZIP.
- A repeatable diagnostic process following the model **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — pinpoint the blocker, apply one documented fix, re-test the affected apps monitor-first, confirm engaging, resume the rollout.

Delivered as a **Complete Solution Package (PDF + Word + scripts)**.

Outcome

A previously stuck Defender for Cloud Apps rollout moving again — the blocker diagnosed against the signal chain, the targeted documented fix applied and verified on the affected apps/policies, and a support-ready bundle ready if the root cause turns out to be a Microsoft service issue.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

TBMMCA-ConditionalAccess.ps1

Purpose: Session policies never engage unless an Entra Conditional Access policy with the "Conditional Access App Control" session control applies to the session. This script enumerates (read-only) every CA policy whose session controls use App Control, reports its enabled state, the target users/groups and apps, and any conditions that might exclude the affected users — pinpointing a session-control blocker. Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TBMMCA-ConditionalAccess.ps1 Requires Microsoft.Graph.Authentication and Policy.Read.All consent. EXPECTED RESULTS A per-policy report (name, state, target apps/users) and a verdict on whether a correctly scoped, enabled App Control policy exists, written to MDCA-conditionalaccess.json. .NOTES White-label. Read-only — reads Conditional Access policy definitions only; changes nothing. VERIFY the intended policy 'Use Conditional Access App Control - Pilot Apps - Diagnose' against your tenant; App Control behavior evolves. > [CmdletBinding()] param([string]$OutputPath = ".\MDCA-conditionalaccess.json")

How to run: powershell -ExecutionPolicy Bypass -File .\TBMMCA-ConditionalAccess.ps1 Requires Microsoft.Graph.Authentication and Policy.Read.All consent. EXPECTED RESULTS A per-policy report (name, state, target apps/users) and a verdict on whether a correctly scoped, enabled App Control policy exists, written to MDCA-conditionalaccess.json. .NOTES White-label. Read-only — reads Conditional Access policy definitions only; changes nothing. VERIFY the intended policy 'Use Conditional Access App Control - Pilot Apps - Diagnose' against your tenant; App Control behavior evolves. > [CmdletBinding()] param([string]$OutputPath = ".\MDCA-conditionalaccess.json")

TBMMCA-Discovery.ps1

Purpose: Read-only discovery for the "Technical Blocker Mitigation — Defender for Cloud Apps" solution. Captures the MDCA signal chain: the licensed plan (indicative), whether cloud-app activity signal is arriving (CloudAppEvents presence — a proxy for working connectors), a device-based discovery hint (DeviceNetworkEvents presence), and a summary of Conditional Access policies that use App Control. Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TBMMCA-Discovery.ps1 Requires the Microsoft.Graph modules. Connects read-only with Connect-MgGraph and prompts for consent to read scopes only (ThreatHunting.Read.All, Policy.Read.All, Organization.Read.All). EXPECTED RESULTS A JSON snapshot (MDCA-blocker-discovery.json) plus a console summary. Read it top-down against the signal chain; the earliest broken link is the blocker. .NOTES White-label. Read-only. Reports on state/telemetry only; changes nothing. Connector status and Cloud Discovery dashboards are authoritative in security.microsoft.com. Licensing facts are indicative — verify current availability in your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\MDCA-blocker-discovery.json", [int]$LookbackHours = 24)

How to run: powershell -ExecutionPolicy Bypass -File .\TBMMCA-Discovery.ps1 Requires the Microsoft.Graph modules. Connects read-only with Connect-MgGraph and prompts for consent to read scopes only (ThreatHunting.Read.All, Policy.Read.All, Organization.Read.All). EXPECTED RESULTS A JSON snapshot (MDCA-blocker-discovery.json) plus a console summary. Read it top-down against the signal chain; the earliest broken link is the blocker. .NOTES White-label. Read-only. Reports on state/telemetry only; changes nothing. Connector status and Cloud Discovery dashboards are authoritative

in security.microsoft.com. Licensing facts are indicative — verify current availability in your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\MDCA-blocker-discovery.json", [int]$LookbackHours = 24)

TBMMCA-Report.ps1

Purpose: Read-only reporting for the "Technical Blocker Mitigation — Defender for Cloud Apps" solution. Compares the discovery baseline (MDCA-blocker-discovery.json) with a fresh live read of the same signal-chain links (activity signal, App Control policy state) and prints a before/after summary — the evidence the blocker moved from broken to cleared. Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TBMMCA-Report.ps1 -Baseline .\MDCA-blocker-discovery.json Requires Microsoft.Graph.Authentication with ThreatHunting.Read.All and Policy.Read.All consent. EXPECTED RESULTS A table of each link's baseline vs current state and an overall CLEARED / STILL-BLOCKED verdict, written to MDCA-blocker-report.json. .NOTES White-label. Read-only. Reads state/telemetry only; changes nothing. > [CmdletBinding()] param([string]$Baseline = ".\MDCA-blocker-discovery.json", [string]$OutputPath = ".\MDCA-blocker-report.json", [int]$LookbackHours = 24)

How to run: powershell -ExecutionPolicy Bypass -File .\TBMMCA-Report.ps1 -Baseline .\MDCA-blocker-discovery.json Requires Microsoft.Graph.Authentication with ThreatHunting.Read.All and Policy.Read.All consent. EXPECTED RESULTS A table of each link's baseline vs current state and an overall CLEARED / STILL-BLOCKED verdict, written to MDCA-blocker-report.json. .NOTES White-label. Read-only. Reads state/telemetry only; changes nothing. > [CmdletBinding()] param([string]$Baseline = ".\MDCA-blocker-discovery.json", [string]$OutputPath = ".\MDCA-blocker-report.json", [int]$LookbackHours = 24)

TBMMCA-SupportBundle.ps1

Purpose: Read-only packaging for the "Technical Blocker Mitigation — Defender for Cloud Apps" solution. When the plan is assigned, the connector re-consented, the discovery source confirmed, and the CA App Control policy enabled+scoped, yet the blocker persists (likely a Microsoft service issue), this assembles the bundle 'MDCA Support Bundle - <case> - Escalate': the discovery snapshot, the CA App Control analysis, the affected apps/policies, tenant context, and timestamps. Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TBMMCA-SupportBundle.ps1 -Case "MSCASE-12345" -AffectedApps "Salesforce,ServiceNow" Point it at the artifacts you already produced: .\TBMMCA-SupportBundle.ps1 -Case "MSCASE-12345" -Discovery .\MDCA-blocker-discovery.json -CAPolicy .\MDCA-conditionalaccess.json EXPECTED RESULTS A single folder 'MDCA-Support-Bundle-<case>' with a manifest and copies of the read-only artifacts, ready to attach to a Microsoft support case. .NOTES White-label. Read-only — collects existing read-only artifacts; changes nothing. Do not place the bundle in shared/unsecured storage; it contains tenant/app identifiers. > [CmdletBinding()] param([Parameter(Mandatory)][string]$Case, [string]$AffectedApps = "", [string]$Discovery = ".\MDCA-blocker-discovery.json", [string]$CAPolicy = ".\MDCA-conditionalaccess.json", [string]$Report = ".\MDCA-blocker-report.json")

How to run: powershell -ExecutionPolicy Bypass -File .\TBMMCA-SupportBundle.ps1 -Case "MSCASE-12345" -AffectedApps "Salesforce,ServiceNow" Point it at the artifacts you already produced: .\TBMMCA-SupportBundle.ps1 -Case "MSCASE-12345" -Discovery .\MDCA-blocker-discovery.json -CAPolicy .\MDCA-conditionalaccess.json EXPECTED RESULTS A single folder 'MDCA-Support-Bundle-<case>' with a manifest and copies of the read-only artifacts, ready to attach to a Microsoft support case. .NOTES White-label. Read-only — collects existing read-only artifacts; changes

nothing. Do not place the bundle in shared/unsecured storage; it contains tenant/app identifiers. > [CmdletBinding()] param([Parameter(Mandatory)][string]$Case, [string]$AffectedApps = "", [string]$Discovery = ".\MDCA-blocker-discovery.json", [string]$CAPolicy = ".\MDCA-conditionalaccess.json", [string]$Report = ".\MDCA-blocker-report.json")

TBMMCA-Validate.ps1

Purpose: Read-only validation for the "Technical Blocker Mitigation — Defender for Cloud Apps" solution. Runs the saved advanced-hunting logic 'MDCA Connector Blocker Check - Tenant - Diagnose' via Microsoft Graph Security to confirm cloud-app activity signal for the affected apps — evidence that connectors are connected and (for session control) that proxied sessions are recorded. Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TBMMCA-Validate.ps1 -AppNameLike "Salesforce" Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS PASS/FAIL lines for connector/activity signal. FAIL points you to the matching Troubleshooting section. Cross-check connector status + discovery dashboard + activity log in the portal. .NOTES White-label. Read-only. Advanced-hunting queries only read telemetry; they change nothing. > [CmdletBinding()] param([string]$AppNameLike = "", [int]$LookbackHours = 24)

How to run: powershell -ExecutionPolicy Bypass -File .\TBMMCA-Validate.ps1 -AppNameLike "Salesforce" Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS PASS/FAIL lines for connector/activity signal. FAIL points you to the matching Troubleshooting section. Cross-check connector status + discovery dashboard + activity log in the portal. .NOTES White-label. Read-only. Advanced-hunting queries only read telemetry; they change nothing. > [CmdletBinding()] param([string]$AppNameLike = "", [int]$LookbackHours = 24)